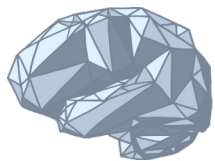




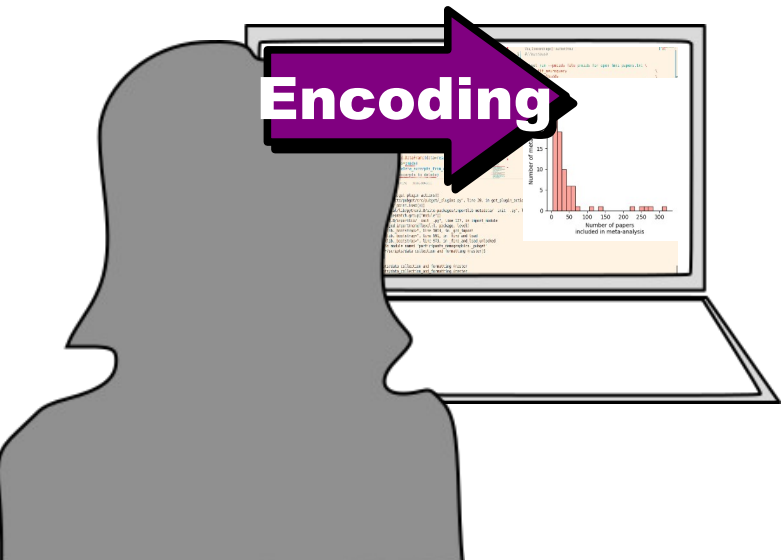
McGill



ORIGAMI
Lab

Introduction to Data Visualization

Part 2: Encoding

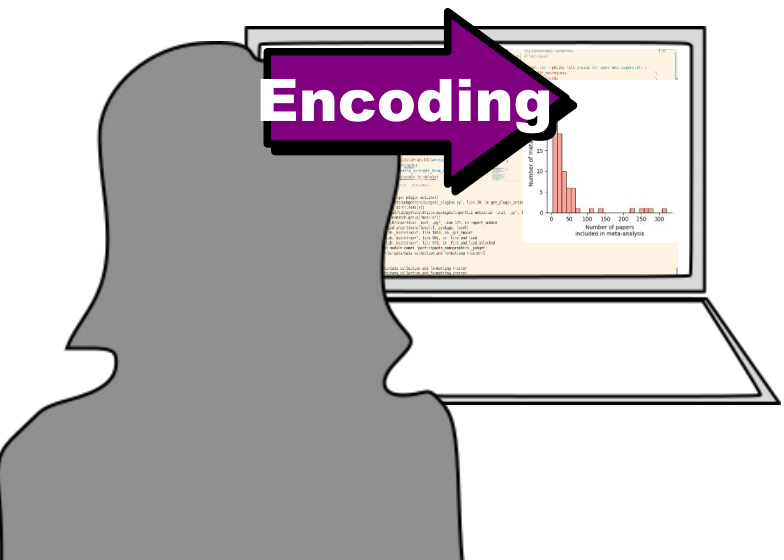


Encoding

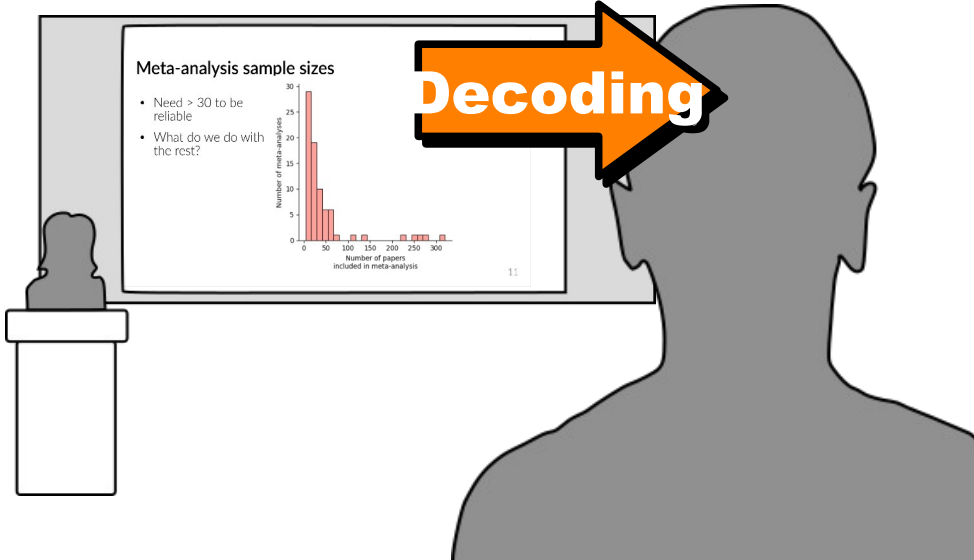
Johanna Bayer
Slides by Kendra Oudyk

Goal

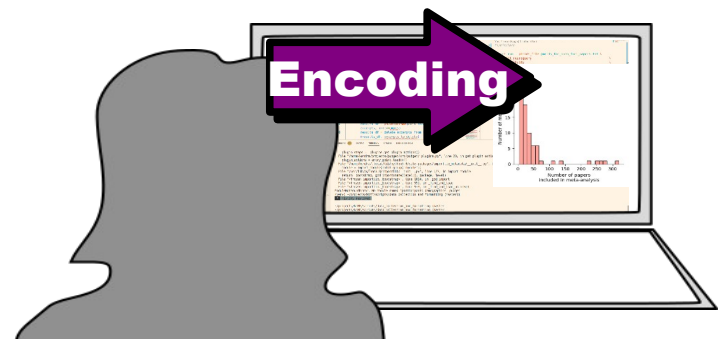
Use principles of visual **encoding** and **decoding** to **efficiently** create visualizations that are **effective** and **reproducible**



Encoding

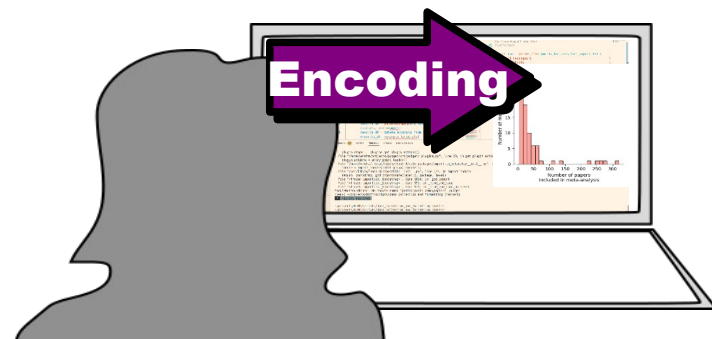
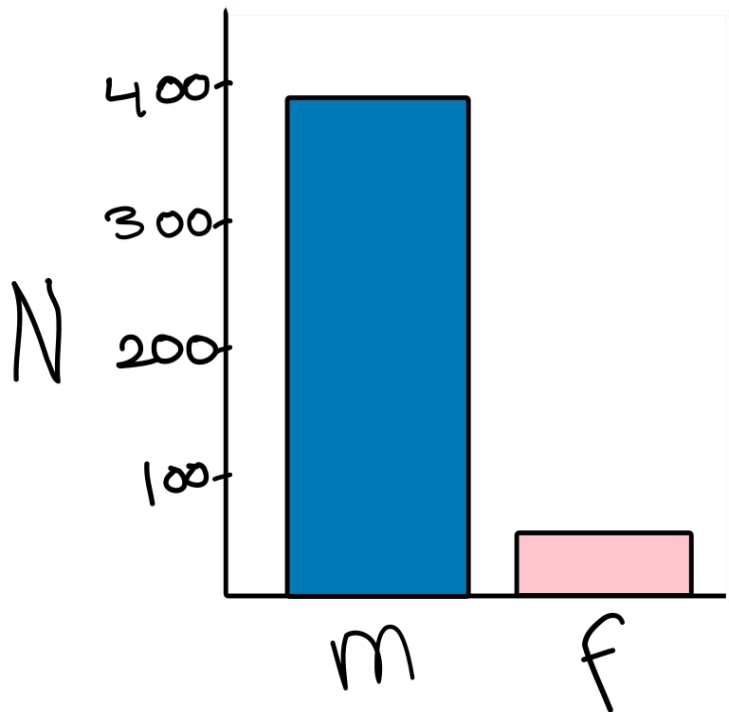


Decoding



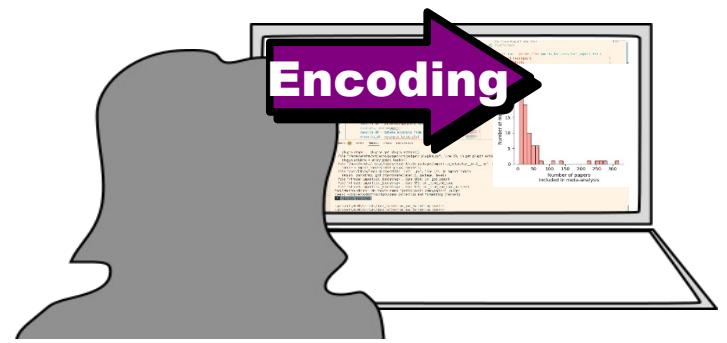
To program a figure efficiently and reproducibly, we should understand

- **Some computer graphics**
 - How the computer makes figure
- **Matplotlib basics**
 - What you and matplotlib need to do
- **Higher-level libraries**
 - More complex visualizations



To program a figure efficiently and reproducibly, we should understand

- **Some computer graphics**
 - How the computer makes figure
- **Matplotlib basics**
 - What you and matplotlib need to do
- **Higher-level libraries**
 - More complex visualizations



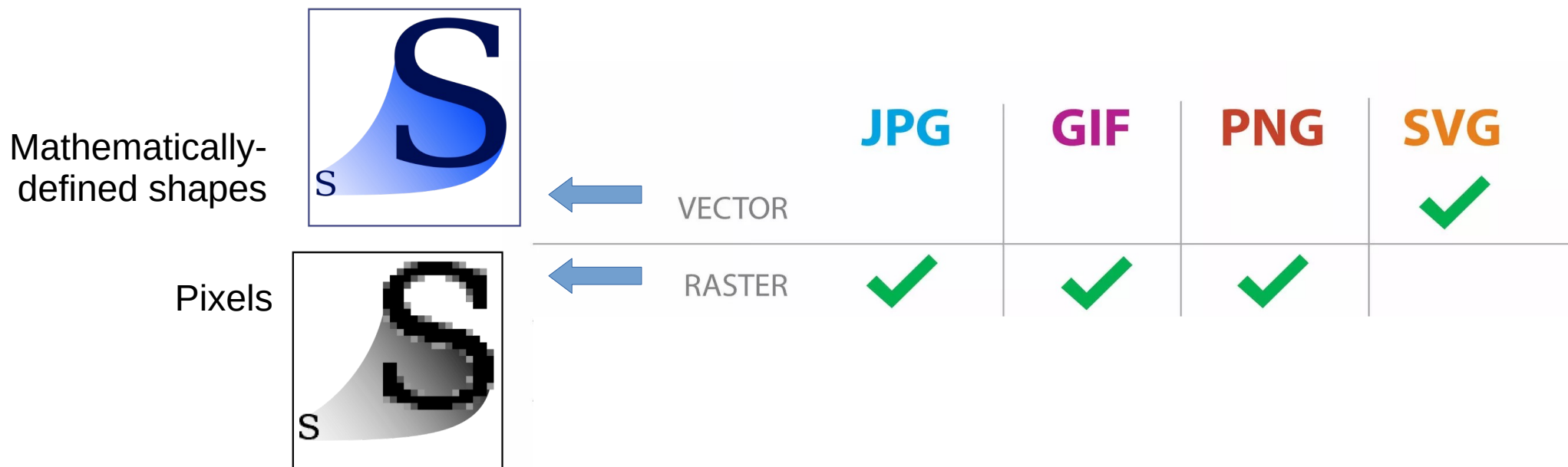
To program a figure efficiently and reproducibly, we should understand

- **Some computer graphics**
 - How the computer makes figure
- **Matplotlib basics**
 - What you and matplotlib need to do
- **Higher-level libraries**
 - More complex visualizations

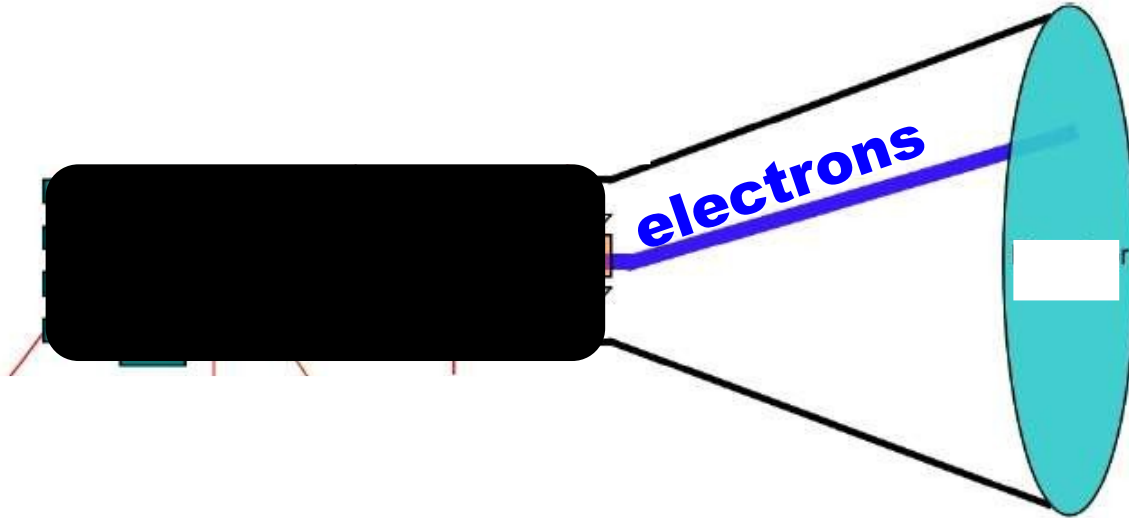
File formats

JPG | **GIF** | **PNG** | **SVG**

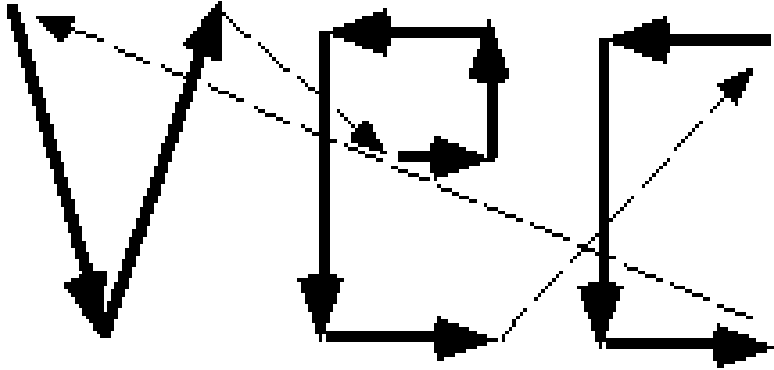
File formats



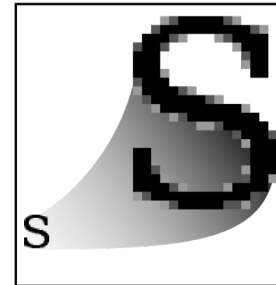
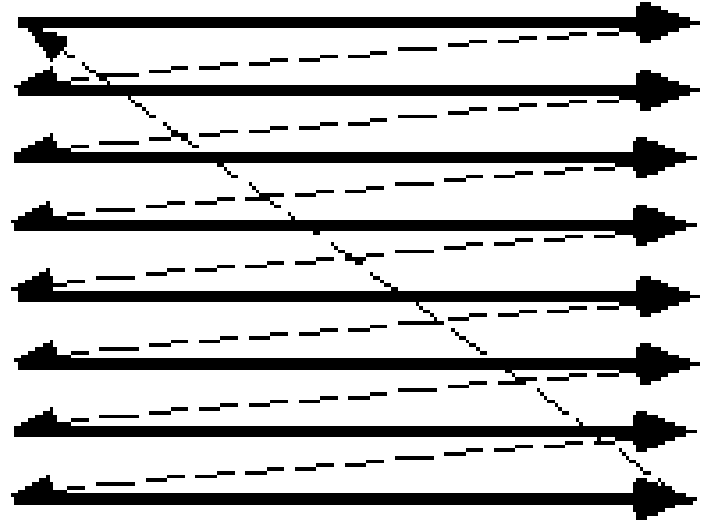
Cathode ray tube



Vector scan



Raster scan

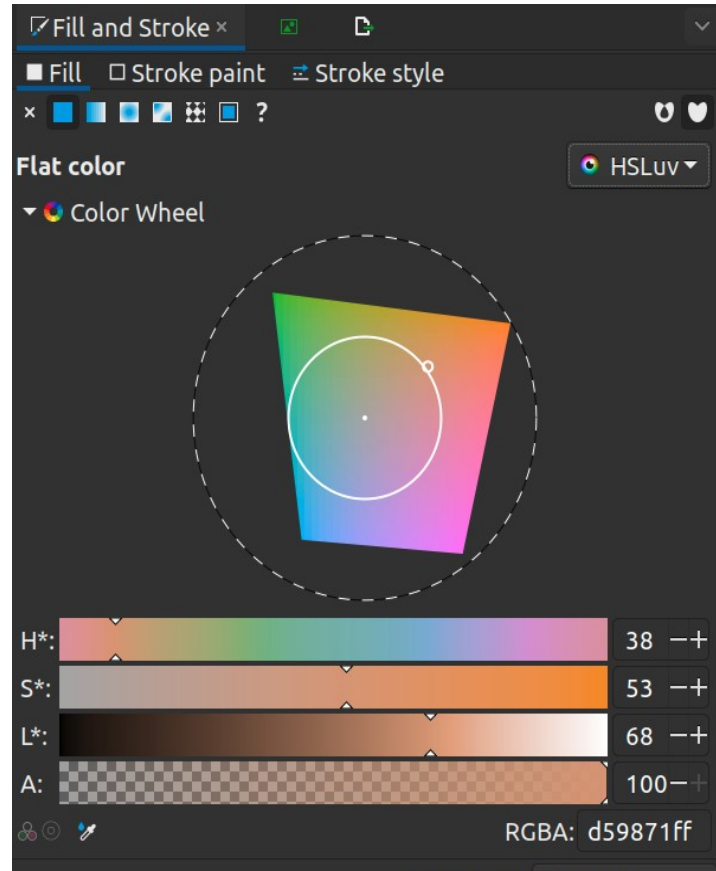


Color: salient but complicated

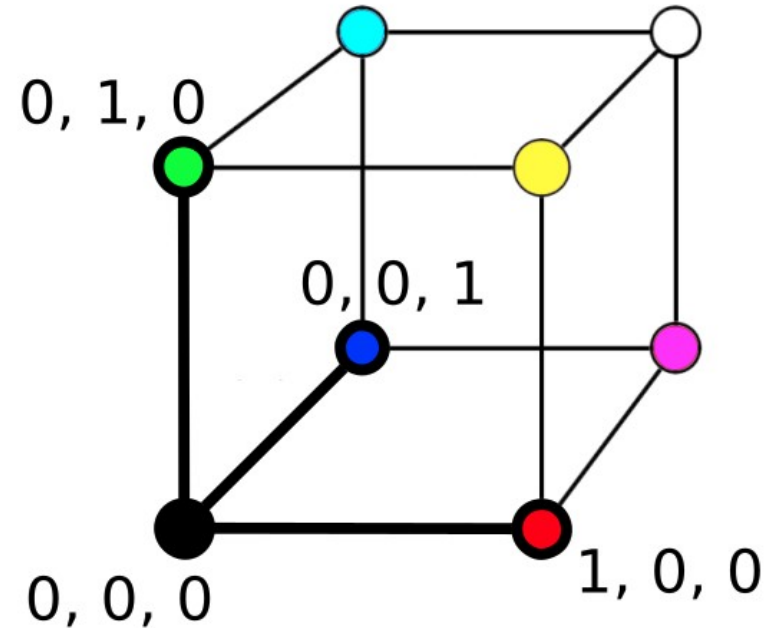
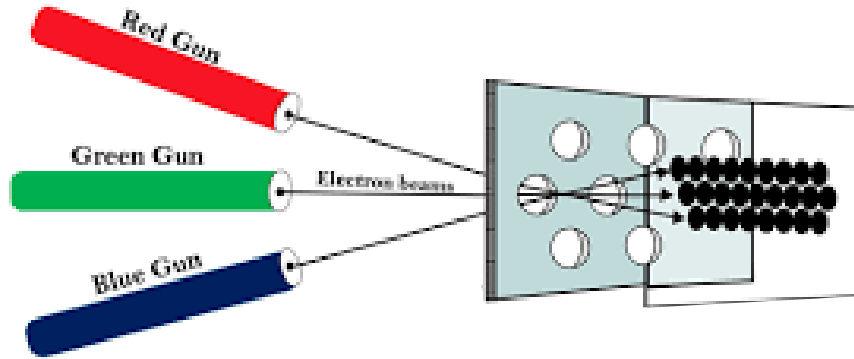


HSLuv: how we talk about color perception

Hue
Saturation
Lightness / Luminance



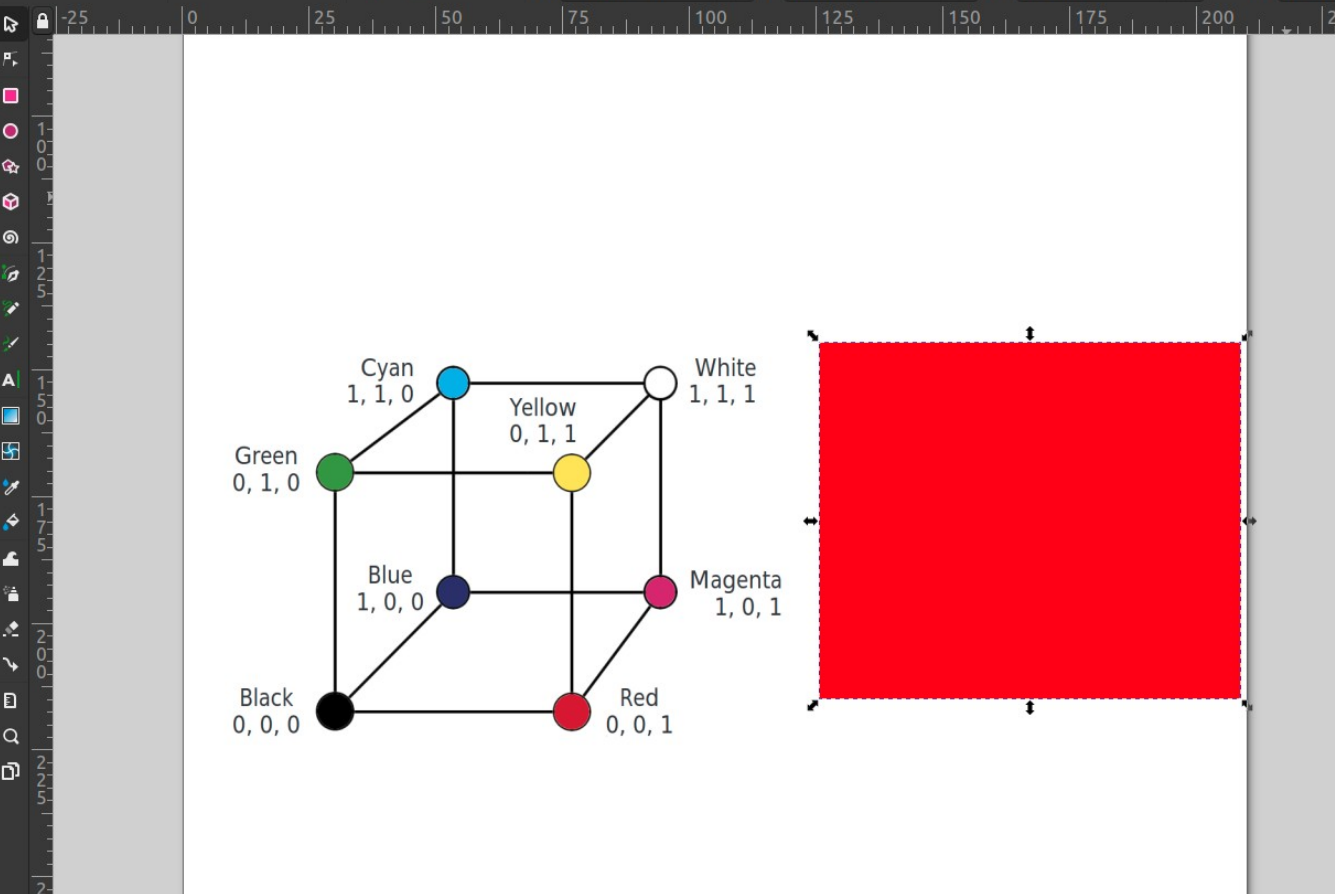
RGB: How a computer produces color



File Edit View Layer Object Path Text Filters Extensions Help



X: 125.591 Y: 144.646 W: 83.250 H: 70.246



Fill and Stroke

Fill Stroke paint Stroke style



Flat color

RGB

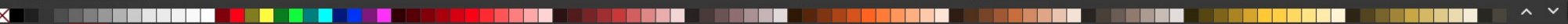
R: 255
G: 0
B: 0
A: 100

RGBA: ff0000ff

Blend mode: Normal

Blur (%) 0.0

Opacity (%) 100.0



Fill: Red Stroke: None 0.784 Layer 1 Rectangle in layer Layer 1. Click selection again to toggle scale/rotation handles.

X: 217.86 Y: 115.94 Z: 122% R: 0.00°

←→↺

https://matplotlib.org/stable/tutorials/colors/colors.html

90%★

🔍🌙stable

📄📁🔗🐙🐦

matplotlib

Plot typesExamplesTutorialsReferenceUser guideDevelopReleases

Section Navigation

Introductory

Intermediate

Advanced

Colors

Specifying colors

Customized Colorbars Tutorial

Creating Colormaps in Matplotlib

Colormap Normalization

Choosing Colormaps in Matplotlib

Text

Toolkits

Provisional

Specifying colors

Color formats

Matplotlib recognizes the following formats to specify a color.

Format	Example
RGB or RGBA (red, green, blue, alpha) tuple of float values in a closed interval [0, 1].	(0.1, 0.2, 0.5) (0.1, 0.2, 0.5, 0.3)
Case-insensitive hex RGB or RGBA string.	#0f0f0f #0f0f0f80
Case-insensitive RGB or RGBA string equivalent hex shorthand of duplicated	#abc as #aabbcc #fb1 as #ffbb11

On this page

Color formats

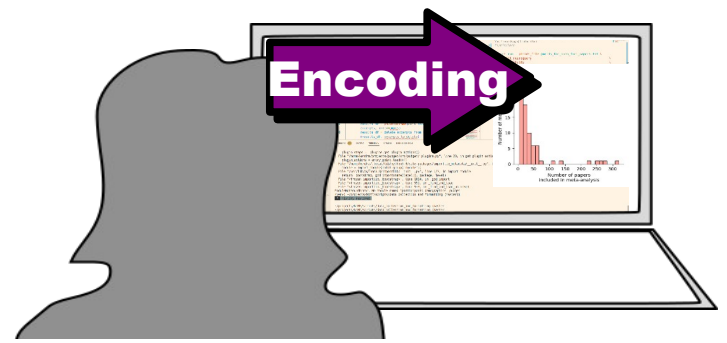
Transparency

"CN" color selection

Comparison between X11/CSS4 and xkcd colors

TL;DR

- Images have different properties/features because of **how they're made**
- The way a computer produces color (**RGB**) is **not intuitive**
 - You'll probably pick colors using named colors or an HSL tool



To program a figure efficiently and reproducibly, we should understand

- **Some computer graphics**

- How the computer makes figure

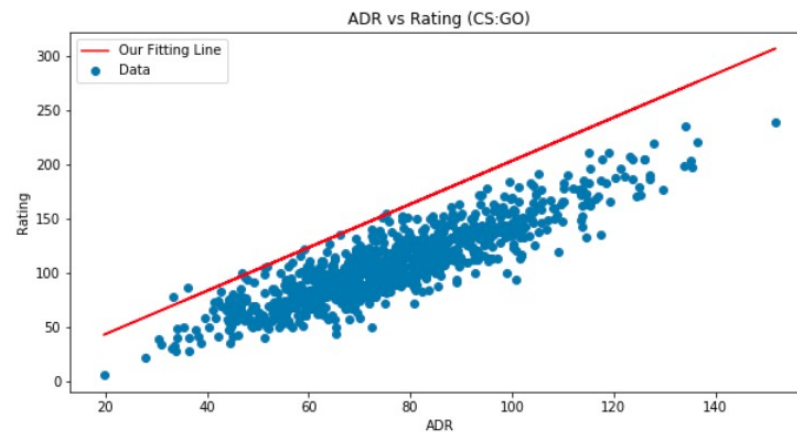
- **Matplotlib basics**

- What you and matplotlib need to do

- **Higher-level libraries**

- More complex visualizations

27



Share Improve this answer Follow

answered Dec 6, 2017 at 7:00

Akash Akash Kandpal

Join Stack Overflow to find the best answer to your technical question, help others answer theirs.

Sign up with email

 Sign up with Google

 Sign up with GitHub

 Sign up with Facebook

2

Activities

Firefox Web Browser

May 4 2:42 PM

59%

open shot inc

Capability to

Artist tutorial

free clipart p

Art Artist Cra

python - Ad x

https://stackoverflow.com/questions/19125722/adding-a-matplotlib-legend

80%

stackoverflow

About

Products

For Teams

Search...

Log in

Sign up

Home

PUBLIC

Questions

Tags

Users

Companies

COLLECTIVES

Explore Collectives

TEAMS

Stack Overflow for Teams – Start collaborating and sharing organizational knowledge.

Free

Create a free Team

Why Teams?

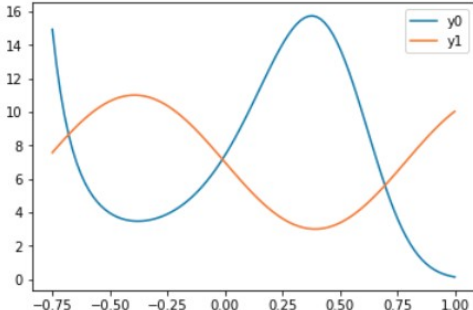
62

You can access the Axes instance (`ax`) with `plt.gca()`. In this case, you can use

```
plt.gca().legend()
```

You can do this either by using the `label=` keyword in each of your `plt.plot()` calls or by assigning your labels as a tuple or list within `legend`, as in this working example:

```
import numpy as np
import matplotlib.pyplot as plt
x = np.linspace(-0.75,1,100)
y0 = np.exp(2 + 3*x - 7*x**3)
y1 = 7-4*np.sin(4*x)
plt.plot(x,y0,x,y1)
plt.gca().legend(('y0','y1'))
plt.show()
```



Join Stack Overflow to find the best answer to your technical question, help others answer theirs.

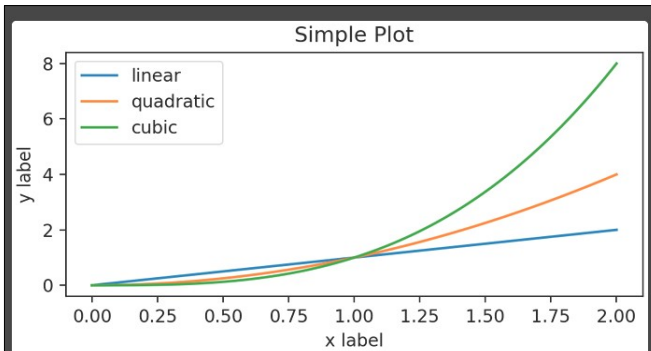
Sign up with email

Sign up with Google

Sign up with GitHub

Sign up with Facebook

Coding styles



Explicit (object-oriented)

```
x = np.linspace(0, 2, 100) # Sample data.  
  
# Note that even in the OO-style, we use `.pyplot.figure` to create the Figure.  
fig, ax = plt.subplots(figsize=(5, 2.7), layout='constrained')  
ax.plot(x, label='linear') # Plot some data on the axes.  
ax.plot(x**2, label='quadratic') # Plot more data on the axes...  
ax.plot(x**3, label='cubic') # ... and some more.  
ax.set_xlabel('x label') # Add an x-label to the axes.  
ax.set_ylabel('y label') # Add a y-label to the axes.  
ax.set_title("Simple Plot") # Add a title to the axes.  
ax.legend()
```

Implicit

```
x = np.linspace(0, 2, 100) # Sample data.  
  
plt.figure(figsize=(5, 2.7), layout='constrained')  
plt.plot(x, label='linear') # Plot some data on the (implicit) axes.  
plt.plot(x**2, label='quadratic') # etc.  
plt.plot(x**3, label='cubic')  
plt.xlabel('x label')  
plt.ylabel('y label')  
plt.title("Simple Plot")  
plt.legend()
```

3 layers of matplotlib

- “**matplotlib.backend_bases.FigureCanvas**

s

is the **area** onto which the figure is drawn

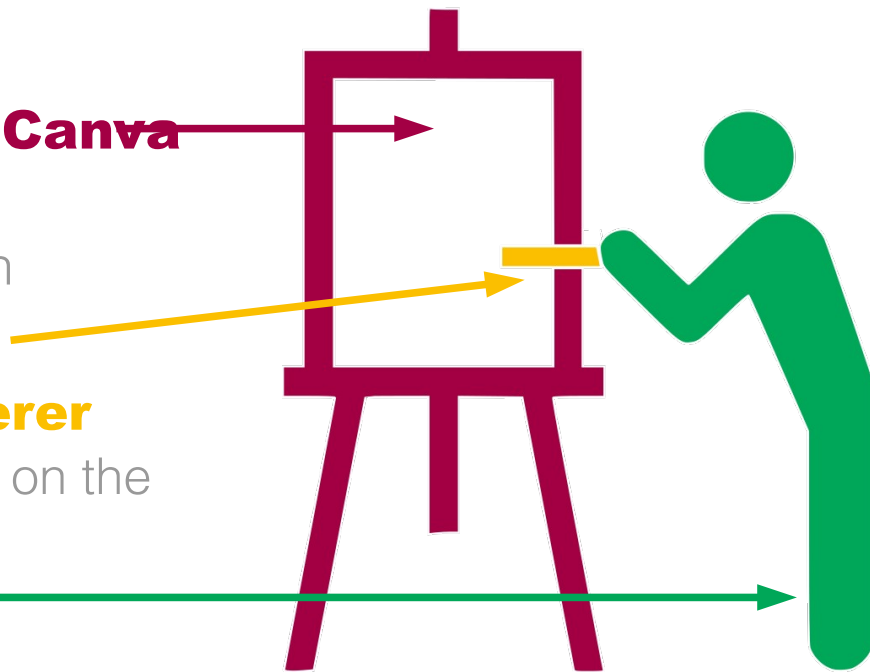
- **matplotlib.backend_bases.Renderer**

is the object which knows how to **draw** on the FigureCanvas

matplotlib.artist.Artist

is the object that knows how to **use a renderer** to paint onto the canvas.”

**What you'll be working
with 95% of the time**



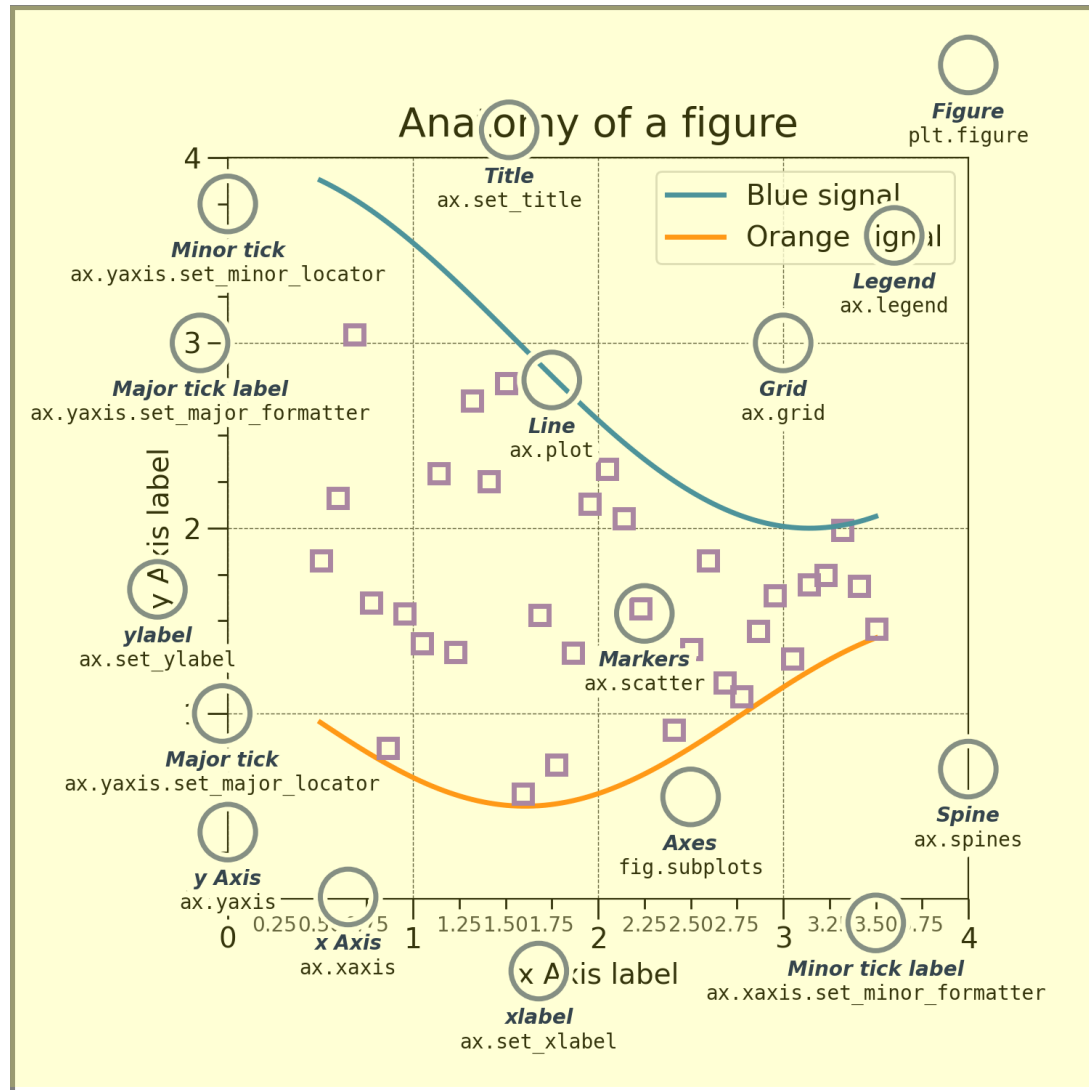
2 types of artists

1) Primitives: things to draw

- Line2D
- Rectangle
- Text
- AxesImage

2) Containers: where to draw them

- Figure
- Axes
- Axis



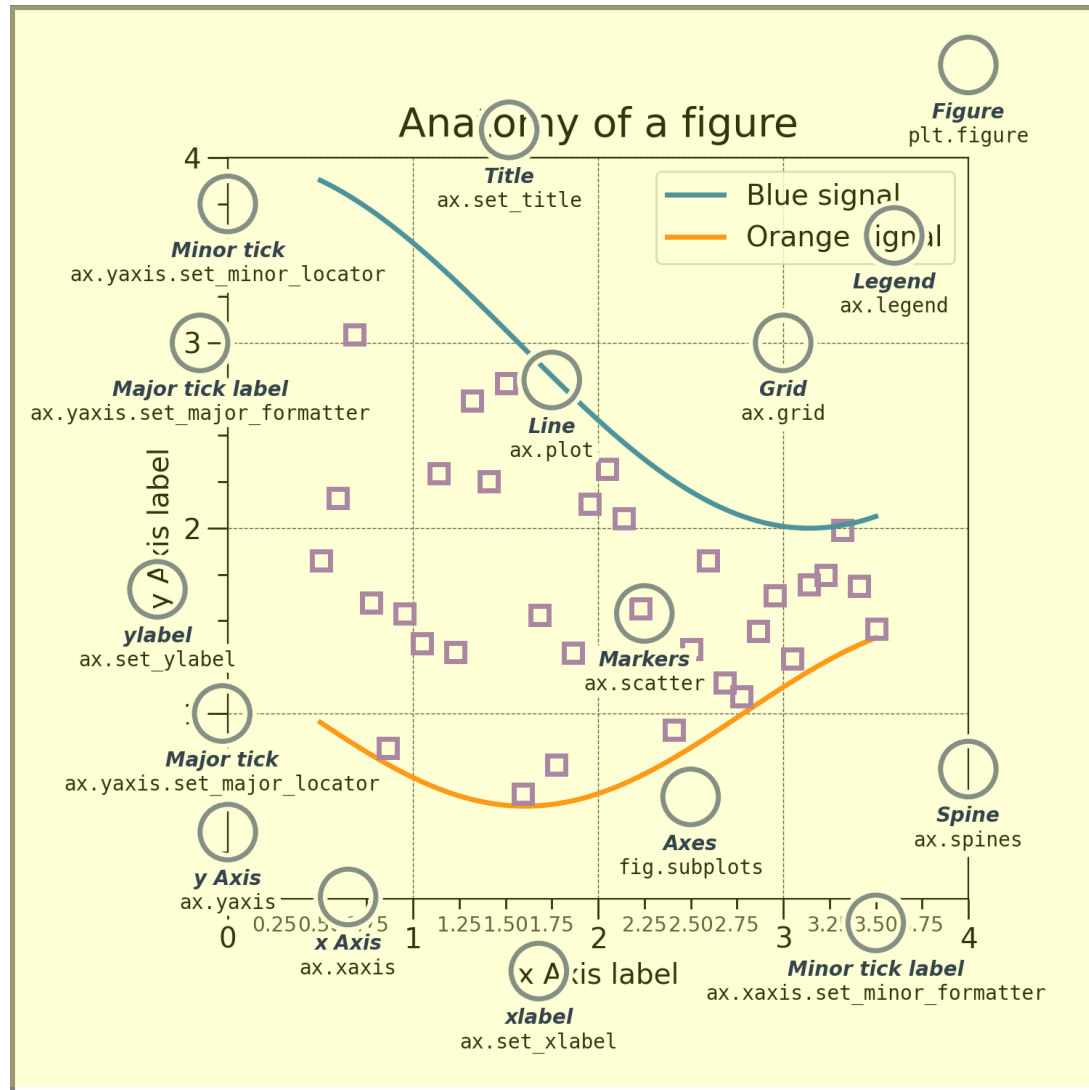
2 types of artists

1) Primitives: things to draw

- Line2D
- Rectangle
- Text
- AxesImage

2) Containers: where to draw them

- **Figure**
- Axes
- Axis



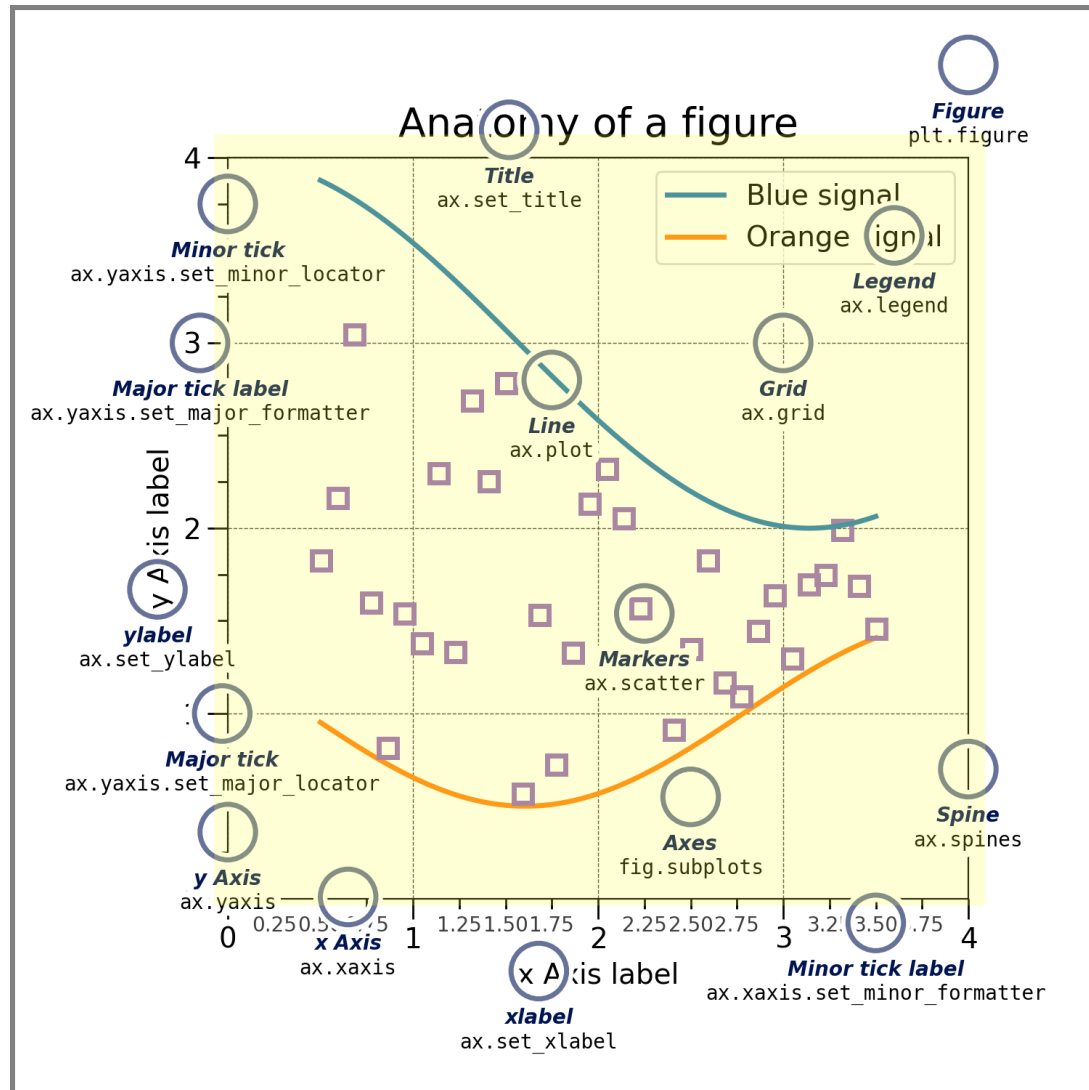
2 types of artists

1) Primitives: things to draw

- Line2D
- Rectangle
- Text
- AxesImage

2) Containers: where to draw them

- Figure
- **Axes**
- Axis



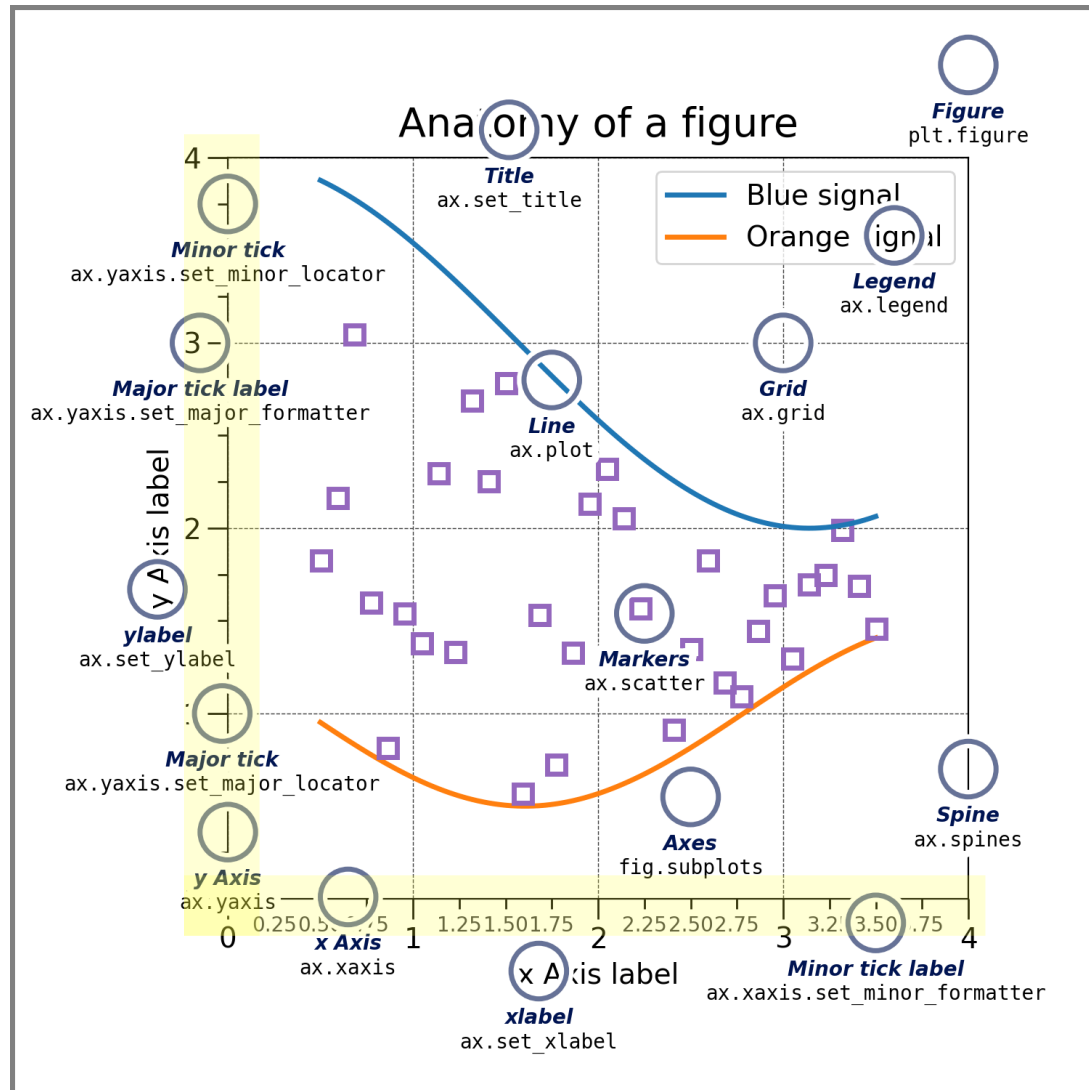
2 types of artists

1) Primitives: things to draw

- Line2D
- Rectangle
- Text
- AxesImage

2) Containers: where to draw them

- Figure
- Axes
- **Axis**



What actually is an Artist??

Artist class

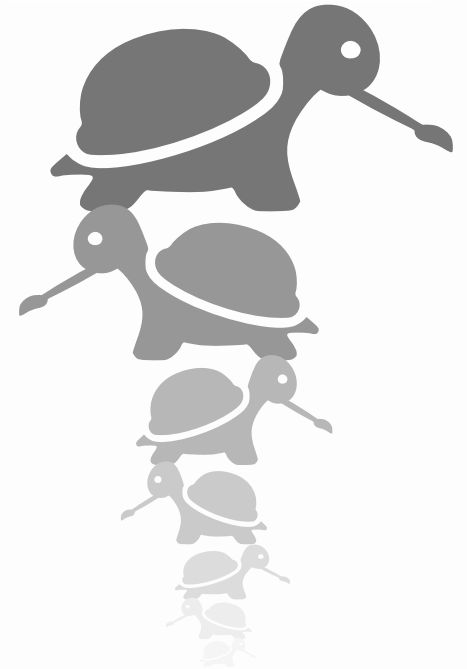
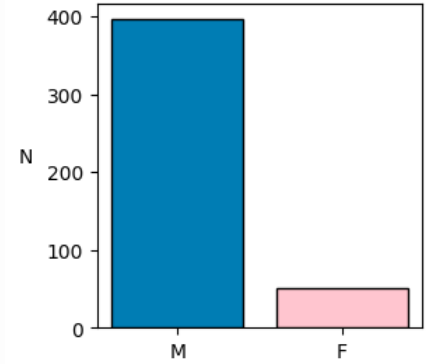
```
class matplotlib.artist.Artist
```

Abstract base class for objects that render into a FigureCanvas.

Typically, all visible elements in a figure are subclasses of Artist.

TL;DR for our purposes,

it's Artists all the way down

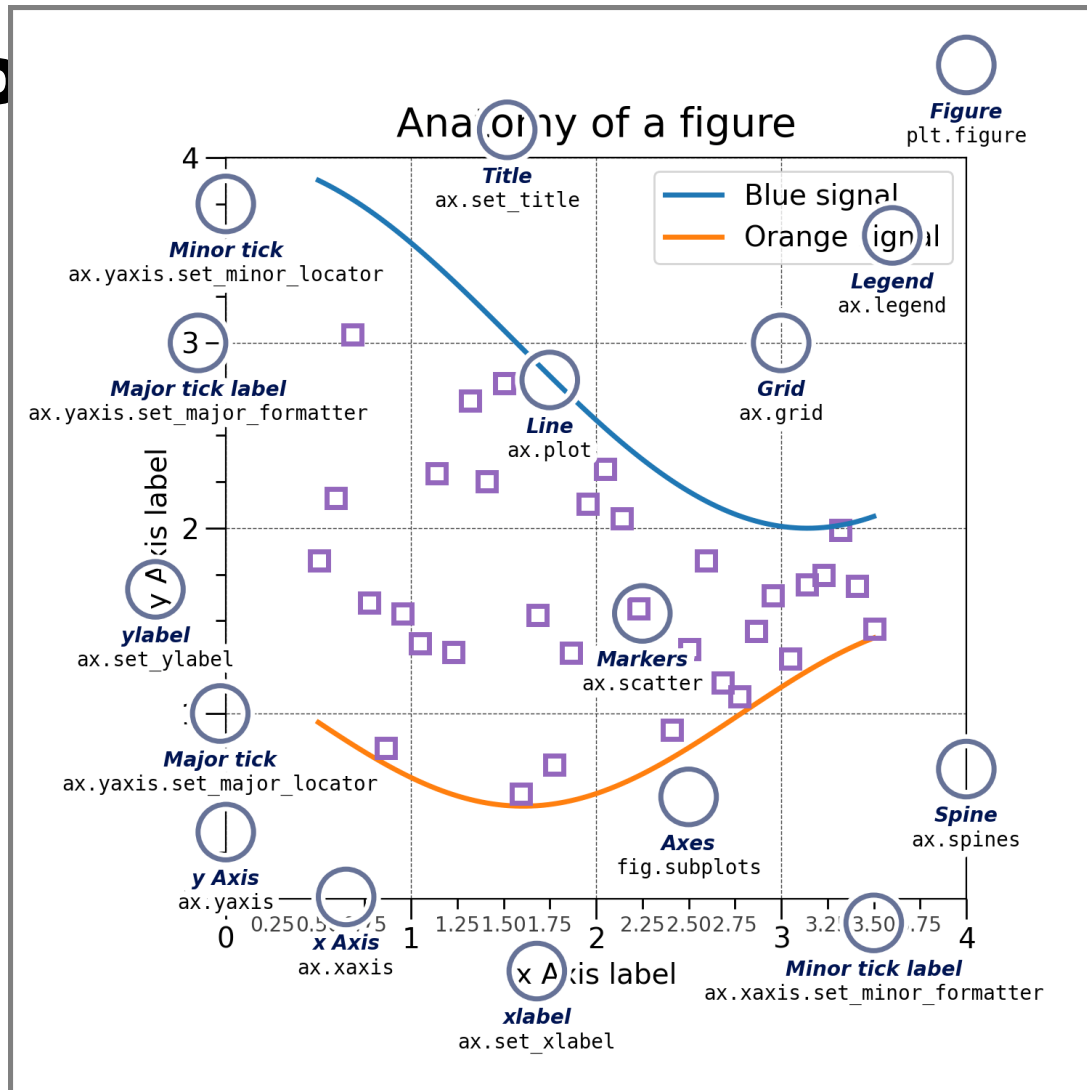


Axes helper methods for plot types

- `ax.plot()` # line graph
- `ax.bar()` # bar graph
- `ax.imshow()` # image

So you don't have to construct figures from shapes

Helper methods for customization



Quick start

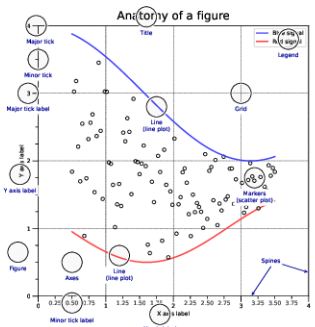
```
import numpy as np
import matplotlib as mpl
import matplotlib.pyplot as plt
```

```
X = np.linspace(0, 2*np.pi, 100)
Y = np.cos(X)
```

```
fig, ax = plt.subplots()
ax.plot(X, Y, color='green')
```

```
fig.savefig("figure.pdf")
fig.show()
```

Anatomy of a figure



Subplots layout

```
subplot[s](rows,cols,...)
fig, axs = plt.subplots(3, 3)
```

```
G = gridspec(rows,cols,...)
ax = G[0,:]
```

```
ax.inset_axes(extent)
```

```
d=make_axes_locatable(ax)
ax = d.new_horizontal('10%')
```

Getting help

- matplotlib.org
- github.com/matplotlib/matplotlib/issues
- discourse.matplotlib.org
- stackoverflow.com/questions/tagged/matplotlib
- https://git.io/matplotlib/matplotlib
- twitter.com/matplotlib
- Matplotlib users mailing list

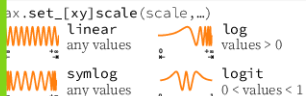
Basic plots



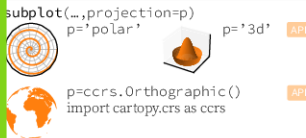
Advanced plots



Scales



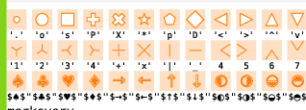
Projections



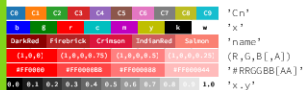
Lines



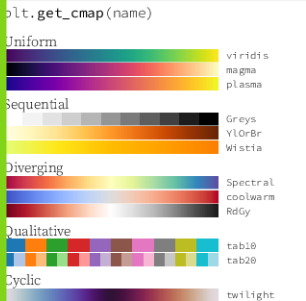
Markers



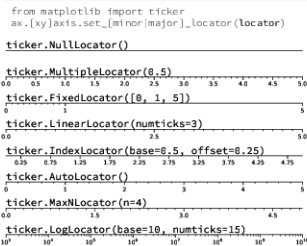
Colors



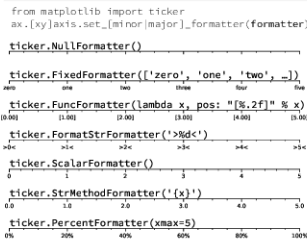
Colormaps



Tick locators

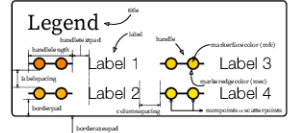


Tick formatters

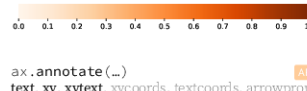


Ornaments

```
ax.legend(...)
handles, labels, loc, title, frameon
```



```
ax.colorbar(...)
mappable, ax, cax, orientation
```



```
ax.annotate(...)
text, xy, xytext, xycoords, textcoords, arrowprops
```

Event handling

```
fig, ax = plt.subplots()
def on_click(event):
    print(event)
fig.canvas.mpl_connect('button_press_event', on_click)
```

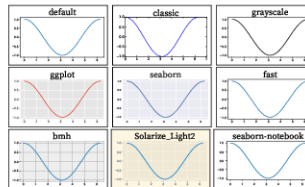
Animation

```
import matplotlib.animation as mpla

T = np.linspace(0, 2*np.pi, 100)
S = np.sin(T)
line, = plt.plot(T, S)
def animate(i):
    line.set_ydata(np.sin(T+i/50))
anim = mpla.FuncAnimation(
    plt.gcf(), animate, interval=5)
plt.show()
```

Styles

```
plt.style.use(style)
```



Quick reminder

```
ax.grid()
ax.set_xlim(vmin, vmax)
ax.set_ylim(vmin, vmax)
ax.set_xticks(ticks, [labels])
ax.set_yticks(ticks, [labels])
ax.set_title(title)
ax.tick_params(width=10, ...)
ax.set_axis_on/off()
```

```
fig.suptitle(title)
fig.tight_layout()
plt.gcf(), plt.gca()
mpl.rc('axes', linewidth=1, ...)
[fig|ax].patch.set_alpha(0)
text=r'$\frac{1}{2}\pi$'
```

Keyboard shortcuts

Ctrl+S	Save	Ctrl+W	Close plot
F	Reset view	F	Fullscreen 0/1
F	View forward	B	View back
P	Pan view	O	Zoom to rect
X	X pan/zoom	Y	Y pan/zoom
G	Minor grid 0/1	C	Major grid 0/1
L	X axis log/linear	L	Y axis log/linear

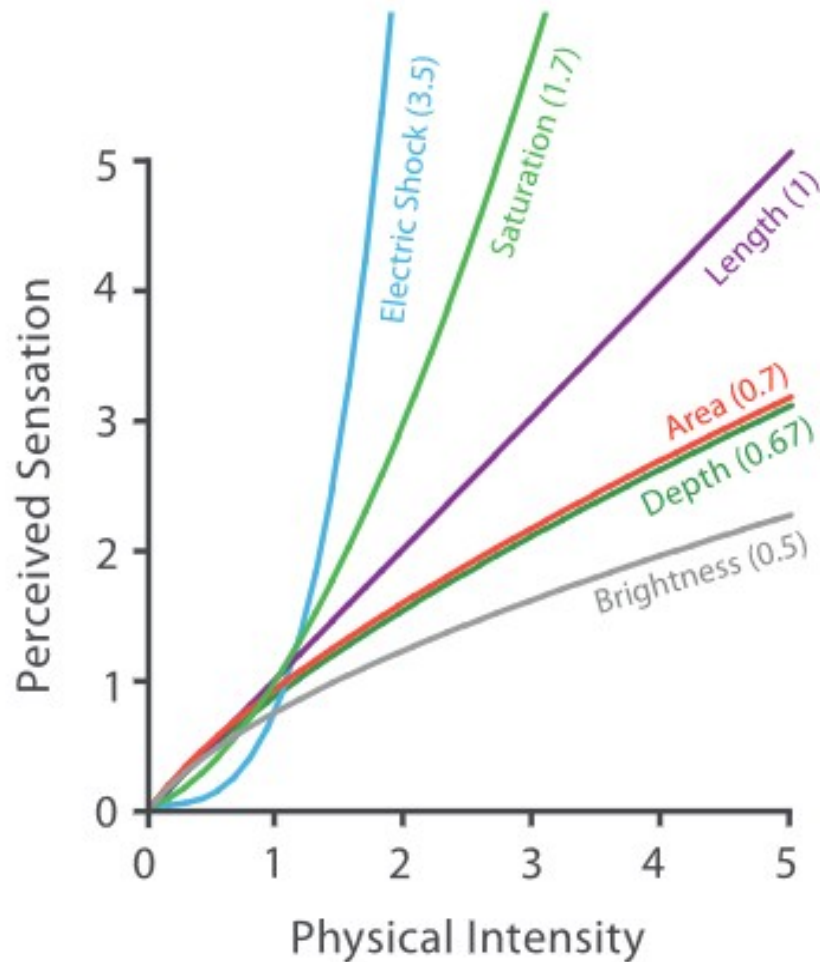
Ten simple rules

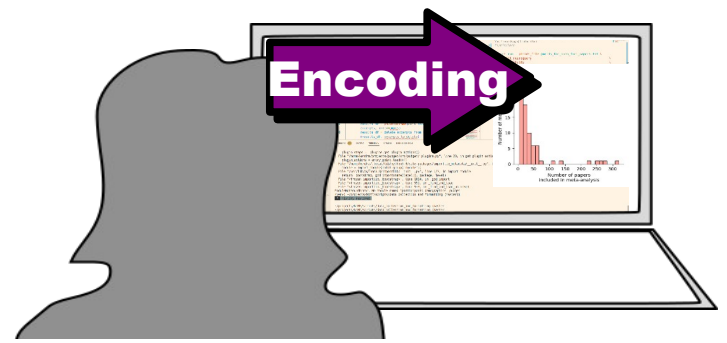
1. Know your audience
2. Identify your message
3. Adapt the figure
4. Captions are not optional
5. Do not trust the defaults
6. Use color effectively
7. Do not mislead the reader
8. Avoid "chartjunk"
9. Message trumps beauty
10. Get the right tool

Live coding

- Replicating this figure

Steven's Psychophysical Power Law: $S = I^N$





To program a figure efficiently and reproducibly, we should understand

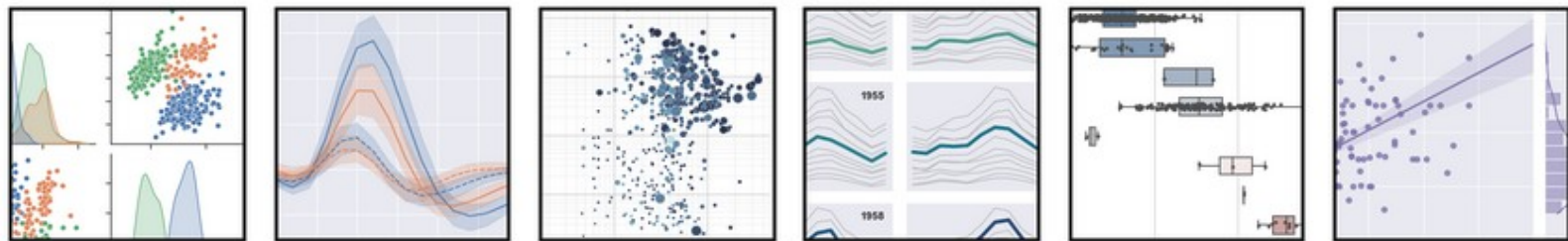
- **Some computer graphics**
 - How the computer makes figure

- **Matplotlib basics**

What you and matplotlib need to do

- **Higher-level libraries**
 - More complex visualizations

seaborn: statistical data visualization



Seaborn is a Python data visualization library based on [matplotlib](#). It provides a high-level interface for drawing attractive and informative statistical graphics.

Features of Seaborn

- Works well with **pandas.DataFrame()** objects
- Default colors are from **perceptually uniform** palettes
- Can **calculate stats** as well as visualize
- More easily visualize **complex data** (e.g, multivariate)



Nilearn

Q Search

Quickstart

Examples

User guide

1. Introduction

2. What is `nilearn`?

3. Using `nilearn` for the first time

4. Machine learning applications to Neuroimaging

5. Decoding and MVPA: predicting from brain images

6. Functional connectivity

7. Plotting brain images

In this section, we detail the general tools to visualize neuroimaging volumes and surfaces with `nilearn`. Nilearn comes with plotting function to display brain maps coming from Nifti-like images, in the `nilearn.plotting` module.

 Code examples

Nilearn has a whole section of the example gallery on plotting.

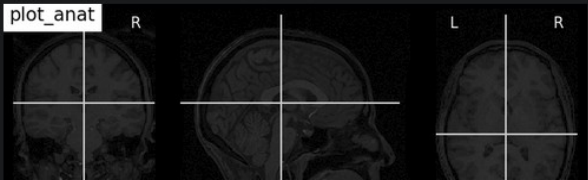
A small tour of the plotting functions can be found in the example [Plotting tools in nilearn](#).

Finally, note that, as always in the `nilearn` documentation, clicking on a figure will take you to the code that generates it.

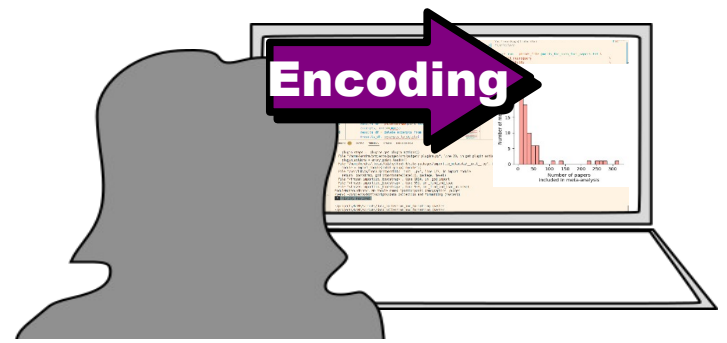
7.1. Different plotting functions

Nilearn has a set of plotting functions to plot brain volumes that are finely tuned to specific applications. Amongst other things, they use different heuristics to find cutting coordinates.

plot_anat



`plot_anat`
Plotting an anatomical image



To program a figure efficiently and reproducibly, we should understand

- **Some computer graphics**
 - How the computer makes figure
- **Matplotlib basics**
 - What you and matplotlib need to do
- **Higher-level libraries**
 - More complex visualizations